

Phase 3: Deletion and Heap Sort — Full Structured Beginner Notes

1. Current Progress

You have already completed the first two phases of the heap curriculum.

What you learned in Phase 1

You learned that a heap is a special binary tree with two important rules:

1. **Shape rule:** the heap must be a complete binary tree.
2. **Ordering rule:** the heap must follow max-heap or min-heap order.

You also learned that this curriculum uses **1-based indexing**:

```
left child  = 2 * i
right child = 2 * i + 1
parent      = i / 2
```

That means:

- The root is at `A[1]`.
- Index `0` is ignored.
- For a node at index `i`, its children and parent can be found using simple formulas.

What you learned in Phase 2

You learned how to insert into a heap.

The main idea of insertion is:

Insert at the next free position first, then shift up.

Insertion moves:

```
bottom → top
leaf → root
```

During insertion, we compare the inserted value with its **parent**.

For a max heap:

If child > parent, move the child upward.

Now Phase 3 teaches the opposite operation.

The main memory rule is:

Insertion moves upward. Deletion moves downward.

2. What This Phase Covers

In Phase 3, you will learn:

1. What deletion means in a heap.
 2. Why heap deletion removes only the root.
 3. Why we cannot leave gaps in a heap.
 4. Why the last element is moved to the root.
 5. What shift-down means.
 6. Why deletion moves downward.
 7. Why we choose the larger child in a max heap.
 8. How to dry run heap deletion.
 9. The C code for deleting from a heap.
 10. How the deletion code works line by line.
 11. How deleted roots are stored at the end of the array.
 12. How heap deletion leads to heap sort.
 13. The full heap sort idea.
 14. Time complexity of deletion.
 15. Time complexity of heap sort.
 16. Space complexity.
 17. Common beginner mistakes.
 18. Important terms and practice questions.
-

3. Big Idea of Phase 3

In Phase 2, you learned how to add a new value into a heap.

Now we learn how to remove the most important value from the heap.

In a **max heap**, the most important value is the largest value.

In a **min heap**, the most important value is the smallest value.

This phase mainly focuses on **max heap deletion**.

For a max heap:

The root contains the largest value.

So deleting from a max heap means:

Delete the root.

4. What Does “Delete” Mean in a Heap?

In a normal array, you can delete any element.

For example, in a normal array you might delete:

A[3]
A[5]
A[7]

But in the standard heap deletion operation taught here, we delete only:

A[1]

Why?

Because A[1] is the root.

In a max heap, the root is the largest value.

Example max heap:

```
      40
     /
    35  15
   / \  /
  30 25 10 5
```

Array form:

Index:	1	2	3	4	5	6	7
Value:	40	35	15	30	25	10	5

Here:

`A[1] = 40`

So deleting from this max heap means deleting:

40

5. Why Do We Delete Only the Root?

A heap is mainly designed for quick access to the highest or lowest priority value.

In a max heap:

Highest priority = largest value = root

In a min heap:

Highest priority = smallest value = root

So the useful deletion operation is root deletion.

This is why heaps are useful for:

- Priority queues
- Heap sort
- Repeatedly getting the maximum
- Repeatedly getting the minimum

6. Why Can't We Just Remove the Root Directly?

Suppose we remove `40` directly from this heap:

Index:	1	2	3	4	5	6	7
Value:	40	35	15	30	25	10	5

After removing 40 :

Index:	1	2	3	4	5	6	7
Value:	_	35	15	30	25	10	5

Now index 1 is empty.

That is a problem because the heap has a gap at the root.

A heap must remain a **complete binary tree**.

That means:

No missing spaces in the active heap area.

So we cannot leave the root empty.

We need to fill the root position.

7. Why Do We Move the Last Element to the Root?

The last element is special because removing it does not create a middle gap.

Example:

Index:	1	2	3	4	5	6	7
Value:	40	35	15	30	25	10	5

The last element is:

$A[7] = 5$

If we remove the last element, the active heap becomes smaller but still compact.

So during deletion:

1. Save the root.
2. Move the last element to the root.
3. Reduce the active heap size.
4. Fix the heap order by shifting down.

This keeps the heap shape valid.

8. Main Steps of Heap Deletion

Heap deletion has these main steps:

1. Save the root value.
2. Move the last element to the root.
3. Treat the heap size as one smaller.
4. Shift the new root downward.
5. Store the deleted root at the end if doing heap sort.

For a max heap:

The moved value may be too small.
So it must move downward until the max-heap rule is restored.

The max-heap rule is:

Parent $\geq$ Child

9. Step 1: Save the Root

The root is the value we are deleting.

In code:

```
val = A[1];
```

Example:

```
A[1] = 40
```

So:

```
val = 40
```

Why save it?

Because we are about to overwrite `A[1]` with the last element.

If we do not save it first, we lose the deleted value.

10. Step 2: Move the Last Element to the Root

After saving the root, we move the last element to index `1`.

In code:

```
A[1] = A[n];
```

Example:

```
A[n] = A[7] = 5
```

So `5` moves to the root.

Before:

```
      40
     /
    35  15
   /  \  /
  30  25 10 5
```

After moving `5` to the root:

```
      5
     /
```

```

      35      15
     /  \   /
    30   25 10

```

Now the shape is okay, but the order is wrong.

11. Step 3: Reduce the Active Heap Size

After moving the last element to the root, the heap size is logically reduced by one.

Original size:

```
n = 7
```

After deleting one root:

```
active heap size = 6
```

The last position can store the deleted value.

Example:

```
Active heap area: indexes 1 to 6
Sorted/deleted area: index 7
```

This is very important for heap sort.

12. Step 4: Shift Down

After moving the last element to the root, the heap order may be broken.

Example:

```

      5
     /
    35      15
   /  \   /
  30   25 10

```


This is not a valid max heap because:

```
5 < 35  
5 < 15
```

In a max heap, the parent should be bigger than or equal to its children.

So **5** must move downward.

This movement is called:

```
shift-down  
percolate-down  
bubble-down
```

These names mean almost the same thing.

Simple meaning:

The value moves downward until it reaches a position where it is greater than or equal to its children.

13. Shift-Down Direction

Deletion moves:

```
top → bottom  
root → leaf
```

During deletion, we compare the moved value with its:

```
children
```

This is the opposite of insertion.

Insertion:

```
Compare with parent and move upward.
```

Deletion:

Compare with children and move downward.

14. Choosing the Larger Child in a Max Heap

When shifting down in a max heap, always choose the **larger child**.

Why?

Because the larger child must be closer to the top.

Example:

```
      5
     /
    35   15
```

Children of 5 are:

35 and 15

The larger child is:

35

Since:

$35 > 5$

we swap 5 with 35.

Result:

```
     35
    /
   5   15
```

If we wrongly swapped with 15, the heap would still be broken because 35 would remain below 5.

So the rule is:

In max heap deletion, swap with the larger child.

15. Full Dry Run: Delete from a Max Heap

Start heap:

Index:	1	2	3	4	5	6	7
Value:	40	35	15	30	25	10	5

Tree:

```
      40
     /
    35  15
   / \  /
  30 25 10 5
```

We delete the root:

Deleted value = 40

Move the last element 5 to the root:

```
      5
     /
    35  15
   / \  /
  30 25 10
```

Now shift down.

16. Dry Run Step 1

Current parent:

5

Children:

35 and 15

Larger child:

35

Compare:

$35 > 5$

So swap 5 and 35.

Tree becomes:

```
      35
     /
    5   15
   / \  /
  30 25 10
```

Array becomes:

35, 5, 15, 30, 25, 10

17. Dry Run Step 2

Now 5 is at index 2.

Children of 5:

30 and 25

Larger child:

30

Compare:

30 > 5

So swap 5 and 30.

Tree becomes:

```
      35
     /
    30  15
   / \ /
  5  25 10
```

Array becomes:

35, 30, 15, 5, 25, 10

Now 5 has no valid children in the active heap.

So shifting stops.

18. Final Result After One Deletion

Deleted value:

40

Active heap:

35, 30, 15, 5, 25, 10

If we store the deleted value at the end, the full array becomes:

35, 30, 15, 5, 25, 10, 40

Here:

35, 30, 15, 5, 25, 10

is the active heap.

And:

40

is the sorted/deleted part.

19. Important Idea: Active Heap vs Sorted Part

After deletion, the array contains two sections:

[active heap area] [sorted part]

Example:

35, 30, 15, 5, 25, 10, 40

Active heap:

35, 30, 15, 5, 25, 10

Sorted part:

40

The sorted part is no longer part of the active heap.

This is the key idea behind heap sort.

20. Delete Function in C

Here is the deletion function:

```
int Delete(int A[], int n) {
    int i, j, val, temp;

    val = A[1];          // save the root value
    A[1] = A[n];         // move last element to root

    i = 1;
    j = 2 * i;           // left child

    while (j < n - 1) {
        // choose the larger child
        if (A[j + 1] > A[j]) {
            j = j + 1;
        }

        // if child is bigger than parent, swap
        if (A[i] < A[j]) {
            temp = A[i];
            A[i] = A[j];
            A[j] = temp;

            i = j;
            j = 2 * j;
        } else {
            break;
        }
    }

    A[n] = val;          // store deleted root at the end
    return val;
}
```

21. Line-by-Line Explanation of Delete

Function header

```
int Delete(int A[], int n)
```

This function deletes the root from the heap.

Here:

- `A[]` is the heap array.
- `n` is the current active heap size.
- The function returns the deleted value.

Declare variables

```
int i, j, val, temp;
```

Meaning:

Variable	Meaning
<code>i</code>	current parent index
<code>j</code>	child index
<code>val</code>	stores the deleted root
<code>temp</code>	helps during swapping

Save the root

```
val = A[1];
```

This saves the value being deleted.

For a max heap, this is the largest value.

Move last element to root

```
A[1] = A[n];
```

This fills the root position using the last element.

This keeps the complete binary tree shape safe.

Start from the root

```
i = 1;
```

We start checking from the root because the replacement value is now at the root.

Find the left child

```
j = 2 * i;
```

Since `i = 1`, initially:

```
j = 2
```

So `j` points to the left child of the root.

Continue while children exist

```
while (j < n - 1)
```

This loop continues while there is a child to compare with.

The heap size is shrinking during deletion, so the boundary matters.

Choose the larger child

```
if (A[j + 1] > A[j]) {  
    j = j + 1;  
}
```

This compares the right child with the left child.

If the right child is larger, `j` moves to the right child.

After this step:

```
j points to the larger child
```

Compare parent with larger child

```
if (A[i] < A[j])
```

If the parent is smaller than the larger child, the max-heap rule is broken.

So we must swap.

Swap parent and child

```
temp = A[i];  
A[i] = A[j];  
A[j] = temp;
```

This moves the larger child upward and the smaller parent downward.

Move down to next level

```
i = j;  
j = 2 * j;
```

Now the moved value is lower in the heap.

So we continue checking from that new position.

Stop if the heap property is correct

```
else {  
    break;  
}
```

If the parent is already greater than or equal to the larger child, the heap is valid.

So we stop.

Store deleted value at the end

```
A[n] = val;
```

This stores the deleted root at the free end position.

This is useful for heap sort.

Return deleted value

```
return val;
```

The function returns the root value that was deleted.

22. Why Store the Deleted Root at the End?

When deleting from a heap for heap sort, we do not throw away the deleted root.

Instead, we place it at the end of the array.

Example:

```
Before deletion:  
40, 35, 15, 30, 25, 10, 5
```

After deleting 40:

```
35, 30, 15, 5, 25, 10, 40
```

The 40 is now at the end.

It is no longer part of the active heap.

It is part of the sorted section.

23. What Is Heap Sort?

Heap sort is a sorting algorithm based on heap deletion.

The idea is:

1. Build a max heap.
2. Delete the root.
3. Store the deleted root at the end.
4. Reduce the heap size.
5. Repeat until the array is sorted.

For a max heap, the root is always the largest active value.

So every deletion removes the largest remaining value.

That largest value is placed at the end.

This creates sorted order from right to left.

Final result:

The array becomes sorted in ascending order.

24. Why Max Heap Sort Gives Ascending Order

In a max heap, the largest value is deleted first.

That largest value is stored at the last position.

Then the next largest value is stored before it.

Then the next largest value is stored before that.

Example idea:

```
[ heap area ..... ] [40]
[ heap area ..... ] [35, 40]
[ heap area .... ] [30, 35, 40]
```

The sorted section grows from right to left.

So the final array becomes ascending from left to right.

25. Heap Sort Example

Suppose the max heap is:

40, 35, 30, 15, 10, 25, 5

First deletion

Delete root:

40

Place it at the end:

[heap area] 40

Second deletion

Delete next root:

35

Place it before 40 :

[heap area] 35, 40

Third deletion

Delete next root:

30

Place it before 35 :

[heap area] 30, 35, 40

Continue until all elements are sorted.

26. Heap Sort Code Structure

Heap sort has two main parts.

Part 1: Build the max heap

```
for (int i = 2; i <= n; i++) {  
    Insert(A, i);  
}
```

This uses repeated insertion to turn the array into a max heap.

Part 2: Delete all roots one by one

```
for (int i = n; i > 1; i--) {  
    Delete(A, i);  
}
```

This repeatedly deletes the root and stores it at the end.

Each time, `i` is the current active heap size.

27. Complete Phase 3 C Program

```
#include <stdio.h>  
  
void Insert(int A[], int n) {  
    int temp = A[n];  
    int i = n;  
  
    while (i > 1 && temp > A[i / 2]) {  
        A[i] = A[i / 2];  
        i = i / 2;  
    }  
  
    A[i] = temp;  
}
```

```

}

int Delete(int A[], int n) {
    int i, j, val, temp;

    val = A[1];
    A[1] = A[n];

    i = 1;
    j = 2 * i;

    while (j < n - 1) {
        if (A[j + 1] > A[j]) {
            j = j + 1;
        }

        if (A[i] < A[j]) {
            temp = A[i];
            A[i] = A[j];
            A[j] = temp;

            i = j;
            j = 2 * j;
        } else {
            break;
        }
    }

    A[n] = val;
    return val;
}

void printArray(int A[], int n) {
    for (int i = 1; i <= n; i++) {
        printf("%d ", A[i]);
    }
    printf("\n");
}

int main() {
    // Index 0 is ignored.
    int A[] = {0, 2, 5, 8, 9, 4, 10};
    int n = 6;

    printf("Original array: ");
    printArray(A, n);

    // Build max heap.

```

```
    for (int i = 2; i <= n; i++) {
        Insert(A, i);
    }

    printf("Max heap: ");
    printArray(A, n);

    // Heap sort.
    for (int i = n; i > 1; i--) {
        Delete(A, i);
    }

    printf("Sorted array: ");
    printArray(A, n);

    return 0;
}
```

28. Expected Program Behavior

The program prints three stages.

Stage 1: Original array

This is the array before heap creation.

Example:

```
2 5 8 9 4 10
```

This is not necessarily a heap.

Stage 2: Max heap

After the build loop, the array becomes a valid max heap.

The root becomes the largest value.

Stage 3: Sorted array

After repeated deletion, the array becomes sorted in ascending order.

29. Important Variables in Phase 3

Variable	Meaning
<code>A[]</code>	The heap array
<code>n</code>	Current active heap size
<code>A[1]</code>	Root of the heap
<code>A[n]</code>	Last element in the active heap
<code>val</code>	Deleted root value
<code>i</code>	Current parent index during shift-down
<code>j</code>	Child index during shift-down
<code>temp</code>	Temporary variable used for swapping

30. Direction Comparison: Insertion vs Deletion

Operation	Starts From	Moves Toward	Compares With
Insertion	Last/new leaf	Root	Parent
Deletion	Root	Leaf	Children

Simple memory:

```
Insertion climbs up.  
Deletion falls down.
```

31. Max Heap vs Min Heap Deletion

This phase focuses on max heap deletion.

But the same idea can be used for min heaps.

Max heap deletion

```
Delete the largest value.  
Choose the larger child during shift-down.
```

Min heap deletion

Delete the smallest value.
Choose the smaller child during shift-down.

For a max heap:

Parent $\geq$ Child

For a min heap:

Parent $\leq$ Child

32. Time Complexity of Deletion

During deletion, the moved value travels downward.

It may move from:

root $\rightarrow$ child $\rightarrow$ grandchild $\rightarrow$ leaf

This path length is the height of the heap.

A heap is a complete binary tree, so its height is:

$O(\log N)$

Therefore:

Delete one root = $O(\log N)$

33. Time Complexity of Heap Sort

Heap sort has two main stages.

Stage 1: Build the heap

Using repeated insertion:

$O(N \log N)$

Because:

- There are about N insertions.
- Each insertion can take $O(\log N)$.

Stage 2: Delete all roots

Repeated deletion:

$O(N \log N)$

Because:

- There are about N deletions.
- Each deletion can take $O(\log N)$.

Total heap sort time

$O(N \log N)$

34. Space Complexity

Heap sort rearranges the same array.

It does not need a second array.

It only uses a few variables:

$i, j, val, temp$

So extra space is:

$O(1)$

This means heap sort is an in-place sorting algorithm.

35. Complexity Summary

Operation	Time Complexity	Reason
Delete one root	$O(\log N)$	The value moves down through heap height
Build heap by repeated insertion	$O(N \log N)$	N insertions, each up to $\log N$
Delete all elements	$O(N \log N)$	N deletions, each up to $\log N$
Heap sort total	$O(N \log N)$	Build heap plus repeated deletion
Extra space	$O(1)$	The same array is rearranged

36. Common Beginner Mistakes

Mistake 1: Deleting from the middle

Wrong idea:

```
Delete A[4]
```

Correct idea:

```
Delete A[1]
```

In standard heap deletion, we delete the root.

Mistake 2: Forgetting to save the root

Wrong:

```
A[1] = A[n];
```

If you do this first, the original root is lost.

Correct:

```
val = A[1];  
A[1] = A[n];
```

Always save the deleted value first.

Mistake 3: Forgetting to move the last element to the root

Some beginners try to shift many nodes upward after deleting the root.

But the correct method is:

```
Move the last element to the root, then shift down.
```

Mistake 4: Swapping with the left child automatically

Wrong idea:

```
Always swap with the left child.
```

Correct idea:

```
Choose the larger child in a max heap.
```

Why?

Because the larger child should move upward.

Mistake 5: Confusing insertion and deletion directions

Insertion:

```
moves upward  
compares with parent
```

Deletion:

moves downward
compares with children

Mistake 6: Forgetting that heap size shrinks

After deletion, the last position is not part of the active heap anymore.

Example:

35, 30, 15, 5, 25, 10, 40

Active heap:

35, 30, 15, 5, 25, 10

Sorted part:

40

Mistake 7: Expecting the heap area to be sorted during heap sort

The active heap is not fully sorted.

Only the sorted section at the end is sorted.

Heap sort gradually creates the sorted section.

37. Key Terms

Term	Simple Meaning
Delete	Remove the root from the heap
Root	The top element, stored at <code>A[1]</code>
Max heap deletion	Removing the largest value
Shift-down	Moving a value downward to restore heap order

Term	Simple Meaning
Percolate-down	Another name for shift-down
Bubble-down	Another name for shift-down
Larger child	The bigger of the two children in a max heap
Active heap	The part of the array still being treated as a heap
Sorted part	The end part of the array containing deleted roots
Heap sort	Sorting by building a heap and repeatedly deleting the root
In-place	Done inside the same array without a second array

38. Rules to Memorize

Rule 1

Heap deletion removes the root.

Rule 2

In a max heap, the root is the largest value.

Rule 3

Save the root before replacing it.

Rule 4

Move the last element to the root.

Rule 5

Shift down to restore heap order.

Rule 6

In a max heap, choose the larger child.

Rule 7

Deleted roots can be stored at the end for heap sort.

Rule 8

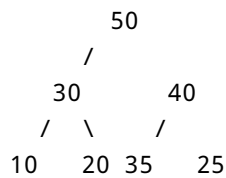
Heap sort uses repeated deletion.

39. Simple Practice Questions

Use this max heap:

Index:	1	2	3	4	5	6	7
Value:	50	30	40	10	20	35	25

Tree:



Questions:

1. Which value is deleted first in max heap deletion?
 2. Why do we move the last element to the root?
 3. What is the last element in this heap?
 4. After moving the last element to root, what direction does it move?
 5. During shift-down in a max heap, do we choose the smaller child or larger child?
 6. What is the time complexity of one deletion?
 7. What is heap sort based on?
 8. Where is the deleted root stored during heap sort?
 9. Does the active heap have to be fully sorted?
 10. What is the extra space complexity of heap sort?
-

40. Practice Answers

1. Which value is deleted first in max heap deletion?

The root is deleted first.

$A[1] = 50$

Answer:

50

2. Why do we move the last element to the root?

Because removing the last element does not create a middle gap.

It keeps the heap as a complete binary tree.

Answer:

To preserve the complete binary tree shape.

3. What is the last element in this heap?

The last index is .

$A[7] = 25$

Answer:

25

4. After moving the last element to root, what direction does it move?

Deletion uses shift-down.

Answer:

Downward

5. During shift-down in a max heap, do we choose the smaller child or larger child?

In a max heap, the larger child should move upward.

Answer:

Larger child

6. What is the time complexity of one deletion?

The value moves down through the heap height.

The heap height is $O(\log N)$.

Answer:

$O(\log N)$

7. What is heap sort based on?

Heap sort is based on repeated root deletion.

Answer:

Build a heap, then repeatedly delete the root.

8. Where is the deleted root stored during heap sort?

The deleted root is stored at the end of the array.

Answer:

At $A[n]$, the free end position.

9. Does the active heap have to be fully sorted?

No.

A heap only needs parent-child order.

Answer:

No, the active heap is not fully sorted.

10. What is the extra space complexity of heap sort?

Heap sort uses the same array.

Answer:

$O(1)$

41. Final Phase 3 Summary

Phase 3 teaches two big ideas:

1. Heap deletion
2. Heap sort

Heap deletion removes the root.

In a max heap:

Root = largest value

The deletion process is:

1. Save the root.
2. Move the last element to the root.
3. Reduce the active heap size.
4. Shift down.
5. Choose the larger child in a max heap.
6. Store the deleted root at the end if sorting.

Heap sort uses this idea repeatedly.

Heap sort process:

Build max heap → delete max → place at end → repeat

Complexities:

Delete one root = $O(\log N)$
Heap sort = $O(N \log N)$
Extra space = $O(1)$

Most important beginner memory:

Insertion moves upward.
Deletion moves downward.
Heap sort repeats deletion.

You are now ready for Phase 4: Heapify, where you will learn a faster way to build a heap.